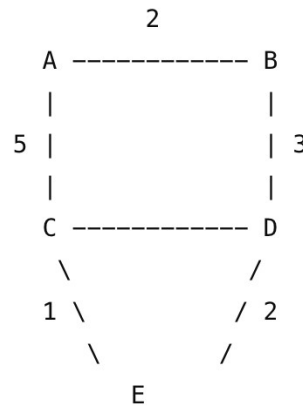


Exercise 2: Link State and Distance Vector Routing

Name: Thanapoom Puchat Student ID: 6510405555 Date: 19 September 2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Note: the C–D link has no label in the diagram; its cost is taken as 1 (consistent with C's distance vector in Question 4: $D = 1$ and $B = 4 = 1 + 3$). Link costs used: A–B 2, A–C 5, B–D 3, C–D 1, C–E 1, D–E 2.

Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

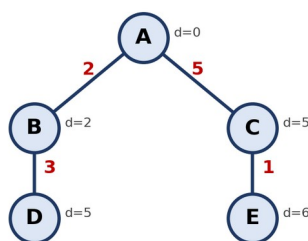
Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	∞	∞
1	B (cost 2)	—	5, A	5, B	∞
2	C (cost 5)	—	—	5, B	6, C
3	D (cost 5)	—	—	—	6, C
4	E (cost 6)	—	—	—	—

Notes: $D(x)$, Prev. = current best cost from A and the previous node on that path. "—" = node already confirmed (permanent). Step 0: A confirmed; its neighbors B (2) and C (5) get tentative costs. Step 1: B has the lowest tentative cost (2); relaxing B–D gives $D = 2 + 3 = 5$ via B. Step 2: C and D tie at 5; C is confirmed first (alphabetical tie-break; the final result is the same either way). Relaxing C–E gives $E = 5 + 1 = 6$ via C; C–D would give $D = 5 + 1 = 6$, which is worse than 5, so D stays 5 via B. Step 3: D confirmed (5); D–E gives $5 + 2 = 7 > 6$, so E stays 6 via C. Step 4: E confirmed (6).

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



Shortest-path tree rooted at A: A-B (2), B-D (3), A-C (5), C-E (1). d = shortest cost from A.

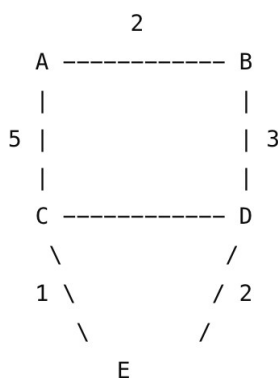
Destination	Shortest Path	Total Cost
B	A → B	2
C	A → C	5
D	A → B → D	5
E	A → C → E	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A → B
C	5	C	A → C
D	5	B	A → B → D
E	6	C	A → C → E

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	$2 + D_B(B) = 2 + 0 = 2$	$5 + D_C(B) = 5 + 4 = 9$	2	B
C	$2 + D_B(C) = 2 + 4 = 6$	$5 + D_C(C) = 5 + 0 = 5$	5	C
D	$2 + D_B(D) = 2 + 3 = 5$	$5 + D_C(D) = 5 + 1 = 6$	5	B
E	$2 + D_B(E) = 2 + 5 = 7$	$5 + D_C(E) = 5 + 1 = 6$	6	C

Bellman-Ford: $D_A(y) = \min \text{ over neighbors } v \text{ of } [c(A,v) + D_v(y)]$, with $c(A,B) = 2$ and $c(A,C) = 5$ (from the topology).

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A → B
C	C	5	A → C
D	B	5	A → B → D
E	C	6	A → C → E

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.

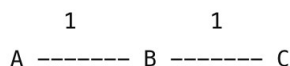
Your answer should describe:

- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
Routers C and D (the two ends of the failed link) detect it locally, e.g. missed Hello/keepalive packets or interface down.	Only C and D detect it locally (same reasons). They set the cost of C–D to infinity. Every other router learns only indirectly, from changed distance vectors.
C and D each originate a new link-state advertisement (LSA/LSP) with a higher sequence number that no longer lists the C–D link. It is flooded reliably out of every interface to all routers.	C and D recompute their own tables using their other neighbors: C → D via E = 1+2 = 3, C → B via E = 1+5 = 6, D → C via E = 2+1 = 3. They send the changed vectors (triggered update) to their neighbors only (E, A, B), and each receiving router recomputes and forwards further changes one hop at a time.
Each router replaces the old LSA with the newer one (sequence number check), deletes the C–D edge from its topology graph, and re-floods it. All link-state databases end up identical again.	Until the new vectors arrive, routers still hold stale information based on old vectors. E.g. C still believes B is at cost 4 via D and D believes C is 1 away. Packets can be dropped or looped and, in bad cases, costs count up slowly ("bad news travels slowly").
Every router independently re-runs Dijkstra on the updated graph. New paths: C → D = C–E–D (3), C → B = C–E–D–B (6). A's paths are unchanged (B 2, C 5, D 5 via B, E 6 via C).	Routers keep exchanging and recomputing with Bellman-Ford. When no vector changes any more the network has converged to the same correct costs (C–D 3 via E, C–B 6 via E, D–C 3 via E; A's routes unchanged).
Convergence is fast: about the flooding time plus one SPF run. Any loop is short-lived, and there is no count-to-infinity.	Convergence takes several update rounds (one hop of information per round), and it is slower and riskier (loops, count-to-infinity) unless mitigations such as split horizon / poison reverse are used.

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

Answer: A stale route can make two routers point at each other. After B–C fails, B has no direct route to C. B then hears A's vector saying "C at cost 2" and does not know that A's path goes through B itself. B therefore picks A as its next hop for C, while A still uses B. A sends C-traffic to B, B sends it back to A, and the packet loops.

2. Show how the estimated distance to C can increase over successive iterations.

Round	New distance to C	Calculation
0 (before failure)	A: 2 via B B: 1 direct	$A = 1 + D_B(C) = 1 + 1 = 2$
1 (B–C fails)	B: 3 via A	B loses its direct route and uses A's stale vector: $1 + 2 = 3$
2	A: 4 via B	A's next hop is B, so A accepts B's new value: $1 + 3 = 4$
3	B: 5 via A	$B: 1 + D_A(C) = 1 + 4 = 5$
4	A: 6 via B	$A: 1 + 5 = 6$
5	B: 7 via A	$B: 1 + 6 = 7$
6	A: 8 via B	$A: 1 + 7 = 8$
...	... keeps growing by 2 per full round	stops only when the cost reaches "infinity" (e.g. 16)

3. What is count-to-infinity?

Answer: Count-to-infinity is the situation in which two (or more) routers keep raising their cost to an unreachable destination by small steps each time they exchange vectors, until the cost reaches a maximum value that is treated as "infinity" (e.g. 16 in RIP). During this time the routing loop persists.

4. Why does it occur in Distance Vector routing?

Answer: Distance Vector routers only see their neighbors' advertised distances, not the actual path or the topology, so they cannot tell that a neighbor's route depends on themselves. Information propagates only hop by hop, so good news travels fast but bad news (a lost route) travels slowly, and each bad update is built on the neighbor's outdated value.

5. Name a mitigation approach.

Answer: Split horizon with poison reverse (A advertises C = infinity to B, because A reaches C through B). Also: a maximum-metric limit (infinity = 16), hold-down timers, triggered updates, or route poisoning.

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Each router learns the whole map, then computes its own routes ("tell everyone about your neighbors").	Each router learns only distances from neighbors and improves its estimates ("tell your neighbors about everyone").
Information maintained	Complete link-state database (all routers, links and costs) plus forwarding table.	Distance vector / routing table (cost and next hop per destination) plus the latest vectors from its neighbors.
Information exchanged	Link-state advertisements (own links and costs), flooded to all routers.	Own distance vector (destination, cost), sent only to directly connected neighbors.
Algorithm	Dijkstra's shortest-path algorithm, run locally; $O(N^2)$ or $O(E \log N)$.	Distributed Bellman-Ford: $D_x(y) = \min_v \{ c(x,v) + D_v(y) \}$, iterative and asynchronous.
Topology knowledge	Global: full knowledge of the topology.	Local: only distances and next hops, no view of the topology.
Routing updates	Event-driven flooding when a link changes (plus infrequent periodic refresh).	Periodic, plus triggered updates on change; whole vector sent to neighbors.

Convergence	Fast (flooding time + SPF computation).	Slow; "bad news travels slowly"; several rounds needed.
Routing loops	Possible only briefly while databases are inconsistent; short-lived.	More likely during convergence and can persist (e.g. count-to-infinity).
Count-to-infinity	Does not occur.	Can occur (mitigated by split horizon, poison reverse, max metric).
Example protocol	OSPF, IS-IS	RIP (also EIGRP, an advanced distance-vector protocol)

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

Choice: Link State (e.g. OSPF or IS-IS with areas), because the network has 500 routers and needs fast convergence, accurate topology knowledge, efficient routing and a hierarchy.

- 1) Fast convergence: a link change is flooded immediately by event-driven LSAs, and every router runs Dijkstra locally on the same database, so recovery takes about flooding time + one SPF run. Distance Vector needs many hop-by-hop rounds and can suffer count-to-infinity.
- 2) Accurate topology knowledge and efficient routing: each router has the full, consistent link-state map. It computes true shortest paths, avoids loops, and can support equal-cost multipath and traffic engineering / cost tuning. Failures are easy to diagnose.
- 3) Hierarchical design: OSPF/IS-IS split 500 routers into areas (backbone area 0 plus regular areas) so flooding and SPF are limited to an area. This keeps database size, CPU and update traffic manageable and allows route summarization between areas.

Disadvantages: higher memory and CPU (each router stores the LSDB and runs Dijkstra), flooding overhead when topology changes are frequent, and more complex design and configuration (area planning, authentication). These are reduced with areas, summarization, SPF throttling and stub areas.

Conclusion: the advantages (fast convergence, loop-free shortest paths, full visibility, hierarchy) outweigh the extra resource cost at this scale, so Link State is the better fit than Distance Vector (RIP), which converges slowly, has a hop-count limit of 15 and does not scale to 500 routers.

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

Information maintained: LS - every router stores the whole link-state database, size $O(N + E)$ (all routers and links), plus the shortest-path tree/forwarding table. DV - a routing table with $O(N)$ entries (cost + next hop), plus the neighbors' vectors, $O(N \cdot d)$ for a router with d neighbors.

Communication overhead: LS - each router's LSA is flooded to all N routers, roughly $O(N \cdot E)$ transmissions for a full refresh, but only when the topology changes (a single link change floods about $O(E)$). DV - every router sends an N -entry vector to each neighbor, $O(N \cdot E)$ entries per round, repeated for many rounds and periodically even when nothing changes.

Computation: LS - Dijkstra per router, $O(N^2)$ (or $O(E \log N)$ with a heap), only when the topology changes. DV - Bellman-Ford, about $O(N \cdot d)$ work per router per update and $O(N \cdot E)$ worst case overall, but this may be repeated many times before the network settles.

Convergence: LS - fast, about the flooding time (proportional to network diameter) plus SPF, and no count-to-infinity. DV - up to $O(N)$ rounds, and unbounded (until "infinity") in count-to-infinity; loops are possible during convergence.

Scalability: LS - scales well because hierarchy (areas) shrinks N and E per database and limits flooding. DV - scales poorly: convergence time and update traffic grow with network size, and a router that advertises wrong distances can mislead the whole network.

Why LS is favored for large intra-domain routing: fast, loop-free convergence, complete topology knowledge (better paths, multipath, traffic engineering), hierarchical areas, and each router computes its own routes so errors stay local. This is why OSPF and IS-IS dominate large networks, and RIP is used only in small ones.